

Agentic Fuzzing of the Parson JSON Parser

Md Polash Islam

Abstract. This report presents an LLM-driven, grammar-based fuzzing campaign against Parson, a lightweight C JSON parser pinned to commit `ba29f4eda9ea7703a9f6a9cf2b0532a2605723c3`. The system starts from an ANTLR JSON grammar, converts the grammar into composable Hypothesis strategies through an iterative LLM loop, executes generated inputs through a sanitizer-enabled C harness, analyzes the results, and uses the feedback to refine subsequent strategies. The official campaign consisted of a 100-input baseline and five 500-input agentic iterations. No crashes, timeouts, sanitizer reports, or nonzero exits were observed within the assigned campaign budget.

1. Design

Grammar and adaptations. The target format is JSON, with `grammar/JSON.g4` used as the formal grammar seed. The grammar describes recursive objects and arrays together with strings, escape sequences, Unicode escapes, numbers, booleans, null, and whitespace. The grammar file itself was kept unchanged; adaptations were made in the generated Hypothesis strategies so that the generator better reflected the behavior observed from Parson. In particular, the valid branch avoids raw control characters, duplicate keys, and zero-with-exponent forms that were frequently rejected during early feedback, while a smaller malformed branch deliberately retains selected near-valid and invalid constructions as parser-boundary probes. The resulting generator therefore preserves recursive structure, nesting, escaping, and numeric edge cases instead of degenerating into unrestricted random strings.

Harness, build, and outcome classification. The C harness reads one input file and calls Parson’s `json_parse_string`. A non-NULL return value is reported as `VALIDJSON`; a NULL return is treated as a clean `INVALIDJSON` rejection. Parsed values are freed after the call. The target and harness are compiled with debug information and AddressSanitizer/UndefinedBehaviorSanitizer using `-g-00-Wall-Wextra-fsanitize=address,undefined`. A nonzero process exit or sanitizer diagnostic is classified as `CRASH`. Each input has a 5-second timeout; following the assignment specification, a timeout is crash-equivalent because a parser that fails to terminate can constitute a denial-of-service vulnerability. A 600-second wall-clock cap is additionally used as a campaign-level safety backstop. The harness is intentionally scoped to Parson’s JSON parser entry point. This keeps the experiment aligned with the target of the assignment: discovering memory-safety failures, undefined behavior, and hangs triggered by JSON parsing. It does not claim coverage of Parson’s separate serialization, validation, persistence, or deep-copy APIs.

Agentic loop and proxy feedback. The agentic loop in `runner/agent_loop.py` constructs each iteration’s prompt from the grammar, the previous generated strategy, and analyzer feedback. The LLM response is saved as an experiment artifact, a candidate `strategy.py` is extracted and validated, and a 40-sample quality gate requires at least 70% Parson-valid generated examples before the full 500-input run. The iteration runner then executes the strategy under the timeout and wall-clock limits and produces analysis used by the next prompt.

Because no source-code coverage signal is available, refinement is guided by three proxy signals: parser acceptance rate, rejection categories, and structural diversity counts such as array items, object keys, nesting, and stress cases. Acceptance rate prevents the limited input budget from being dominated by useless rejection; rejection categories identify constructions that need correction or controlled retention; and structural counts encourage the LLM to preserve recursive and stress-oriented input diversity. These measures are explicitly treated as proxies for exploration, not as claims of actual code coverage.

2. Findings

The baseline used intentionally weak random text to verify the complete pipeline and establish a reference point. Only 3 of 100 baseline inputs were accepted by Parson. The grammar-aware, quality-gated strategies generated by the agentic loop subsequently maintained a validity rate above 92% in every official iteration while retaining a controlled malformed/stress branch.

Run	Total	Valid	Invalid	Crash	Timeout	Valid Rate
Baseline	100	3	97	0	0	3.00%
Iteration 1	500	464	36	0	0	92.80%
Iteration 2	500	473	27	0	0	94.60%
Iteration 3	500	468	32	0	0	93.60%
Iteration 4	500	460	40	0	0	92.00%
Iteration 5	500	463	37	0	0	92.60%

Strategy evolution. Iteration 1 focused on converting the grammar into a practical generator with a high proportion of accepted recursive JSON. Iteration 2 further corrected constructions that were disproportionately rejected, improving the valid rate to 94.60%. Iteration 3 continued this balance while retaining malformed and stress-oriented cases. Iteration 4 shifted more strongly toward structural stress; although its valid rate decreased to 92.00%, structural counts increased substantially, including `array_items:6199` and `object_keys:6168`. Iteration 5 retained this balance rather than optimizing acceptance alone. The evolution therefore reflects a deliberate trade-off: maximize useful parser-reaching inputs while reserving part of the budget for boundary and stress exploration.

Crash findings. No crash-equivalent failure was found. Across the checked campaign there were 0 crashes, 0

timeouts, 0 sanitizer reports, and 0 nonzero exits. Consequently, there are no crash signatures or minimized reproducers to report. The appropriate conclusion is bounded: the executed campaign did not reach a crashing or hanging parser path. It does not establish that Parson is free of defects. A likely reason is that the available 2,500 agentic inputs were finite and the quality gate intentionally concentrated most of the budget on well-formed, parser-reaching JSON rather than indiscriminate malformed input.

The remaining invalid inputs were primarily associated with campaign-specific categories such as `syntax_invalid_for_python_json`, `duplicate_key`, and `zero_with_exponent`. These labels describe how the analysis pipeline categorized generated cases; they are not claims that Parson violates the JSON standard.

Under-tested grammar and parser boundaries. The campaign likely under-explored very large documents, extreme nesting, very long escape-heavy strings, Unicode-heavy combinations, and combinations of numeric edge cases with deep recursive structures. These dimensions are especially relevant for finding resource exhaustion or parser-state bugs but are expensive under the per-input and total-run limits.

3. Challenges, Judgment Calls, and Next Steps

The main engineering challenge was preventing the LLM-generated strategy from spending too much of the limited budget on inputs that Parson immediately rejects. Early generator behavior demonstrated that a grammar-shaped strategy can still be operationally weak if it does not match the target parser closely enough. The 40-sample quality gate and subsequent feedback-driven refinement addressed this problem while preserving a smaller population of malformed probes.

A second judgment call concerned timeouts. The runner records `TIMEOUT` separately so that hangs remain diagnostically visible, but treats them as crash-equivalent in accordance with the assignment. This avoids silently discarding denial-of-service-style parser failures.

Crash deduplication was designed around normalized sanitizer stack frames. `scripts/triage_crashes.py` extracts the top frames, normalizes compiler-generated suffixes, and hashes the normalized stack into a `SIG-...` signature. Repeated manifestations of the same root cause can therefore be grouped instead of being counted as separate bugs. If a crash is reproducible from the generated strategy, `scripts/minimize_crash.py` uses Hypothesis shrinking and then performs standalone verification. These mechanisms were implemented for the required workflow, but no real crash-equivalent case existed to exercise them during the official campaign.

With more time, I would increase targeted stress along depth, document size, escape length, Unicode combinations, and numeric extremes. With coverage feedback available, I would replace structural diversity proxies with coverage deltas and use newly reached branches as the primary refinement signal. A broader Parson API harness could also be developed as a separate experiment, but that would extend beyond the parser-entry scope of this campaign.

4. Reproducibility and Artifacts

The repository contains the complete implementation needed to inspect and reproduce the campaign. The main artifacts are:

- **Grammar:** `grammar/JSON.g4`
- **Harness:** `harness/harness.c`
- **Build script:** `harness/build.sh`
- **Baseline strategy:** `strategies/baseline.py`
- **Agentic loop:** `runner/agentic_loop.py`
- **Iteration runner:** `runner/iteration_runner.py`
- **Result analysis:** `scripts/analyze_results.py`
- **Crash triage:** `scripts/triage_crashes.py`
- **Crash minimization:** `scripts/minimize_crash.py`

Each official iteration retains its prompt, LLM response, generated strategy, results, analysis, and metadata, allowing the evolution of the fuzzer to be inspected and reproduced.

The five LLM iterations used `gpt-5-mini`: 19,620 input tokens and 19,325 output tokens, for 38,945 total tokens and an estimated cost of approximately \$0.04. The per-iteration metadata and aggregate usage records provide the campaign accounting.

Conclusion. The campaign demonstrates an end-to-end agentic fuzzing workflow that transformed a weak random baseline into a grammar-aware, high-validity generator while retaining structural and parser-boundary stress. No crash or timeout was observed within the assigned budget. The result should therefore be interpreted as evidence about this particular campaign, not as a proof of parser correctness. The strongest next step is targeted stress generation, ideally guided by code-coverage feedback when such instrumentation is available.